Dynamical correlation functions (MPS)

Jannis Lutz
(Dated: March 17, 2026)

Matrix product state algorithms are powerful tools to simulate quantum many-body systems. In this work, we use Time-Evolving Block Decimation (TEBD) and Density Matrix Renormalization Group (DMRG) methods to evaluate correlation functions of the form $\left\langle \psi_0 \middle| e^{i\hat{H}t}\hat{S}_i^+ e^{-i\hat{H}t}\hat{S}_j^- \middle| \psi_0 \right\rangle$, where $|\psi_0\rangle$ is the ground state of the 1D transverse-field Ising model. For the system parameters considered, DMRG and TEBD yield accurate results when compared to exact diagonalization as a reference. We then increase the system size and perform discrete Fourier transforms in space and time to compute the corresponding spectral function, providing insight into the excitation spectra at various transverse-field strengths.

I. INTRODUCTION

A. Transverse-field Ising model

The underlying Hamiltonian for our discussion is the one-dimensional transverse-field Ising model

$$\hat{H} = -J\sum_i \hat{\sigma}_i^x \hat{\sigma}_{i+1}^x - g\sum_i \hat{\sigma}_i^z \tag{1}$$

with Pauli matrices $\hat{\sigma}_i^j$, transverse-field strength g, and coupling strength J. The latter will be set to 1 in the following. This model can be solved analytically by performing a Jordan-Wigner transformation, i.e. mapping the spin operators to fermionic operators $\hat{c}_j$:

$$\begin{aligned}\hat{\sigma}_j^z &= \mathbb{1} - 2\hat{c}_j^\dagger \hat{c}_j \\ \hat{\sigma}_j^- &:= \frac{1}{2}\left(\hat{\sigma}_j^x - i\hat{\sigma}_j^y\right) = (-1)^{\sum_{k<j}\hat{c}_k^\dagger \hat{c}_k} \cdot \hat{c}_j^\dagger \\ \hat{\sigma}_j^+ &:= \frac{1}{2}\left(\hat{\sigma}_j^x + i\hat{\sigma}_j^y\right) = (-1)^{\sum_{k<j}\hat{c}_k^\dagger \hat{c}_k} \cdot \hat{c}_j.\end{aligned}$$

Following that, a discrete fourier transform of the form $\hat{c}_j = 1/\sqrt{N}\sum_k e^{-ikj}\hat{c}_k$ is performed which yields the Hamiltonian in momentum representation

$$\hat{H} = \sum_k [2\cos k - 2g]\hat{c}_k^\dagger \hat{c}_k + \text{const.}$$

Ultimately, a Bogoliubov transformation diagonalizes the Hamiltonian $\hat{H} = \sum_k \varepsilon_k \hat{d}_k^\dagger \hat{d}_k$ and we obtain the quasi-particle energy dispersion

$$\varepsilon_k^\pm = \pm 2\sqrt{(\cos k - g)^2 + \sin^2 k} \tag{2}$$

This energy gap closes at $g = J = 1$ and $k = \pi$, indicating the presence of a quantum phase transition at the critical point $g_{\text{crit}} = 1$. The system transitions from a ferromagnetic phase ($g < 1$) to a paramagnetic phase ($g > 1$). [1, 2]

When computing the spectral function later on, we will try to recover the shape of equation (2).

B. Matrix product states (MPS)

A generic many-body quantum state of a spin-$\frac{1}{2}$ system of size N can be written as

$$|\psi\rangle = \sum_{j_1,j_2,\ldots,j_N} \psi_{j_1,j_2,\ldots,j_N} |j_1\rangle \otimes |j_2\rangle \otimes \cdots \otimes |j_N\rangle\,, \tag{3}$$

where $\psi_{j_1,j_2,\ldots,j_N}$ is an order N tensor. The idea of a MPS is to rewrite equation (3) as a product of matrices

$$\begin{aligned}|\psi\rangle &= \sum_{j_1,\ldots,j_N}\sum_{\alpha_2,\ldots,\alpha_N} M^{[1]j_1}_{\alpha_1\alpha_2} M^{[2]j_2}_{\alpha_2\alpha_3}\ldots M^{[N]j_N}_{\alpha_N\alpha_{N+1}} |j_1, j_2, \ldots, j_N\rangle \\ &\equiv \sum_{j_1,\ldots,j_N} M^{[1]j_1} M^{[2]j_2}\ldots M^{[N]j_N} |j_1, j_2, \ldots, j_N\rangle\,.\end{aligned} \tag{4}$$

Each $M^{[n]j_n}$ is a $\chi_n \times \chi_n$ dimensional matrix, where χ_n is denoted as the bond dimension. The α_n's are referred to as "virtual" indices and the j_n's are the so called "physical" indices. The MPS representation can be obtain by successive reshaping and applying singular value decompositions (SVDs) to the initial coefficient tensor $\psi_{j_1,j_2,\ldots,j_N}$.[3]

This ansatz is very useful for numerical simulations. Unlike exact diagonalization methods whose computational cost usually grows exponentially with system size N, MPS algorithms typically scale polynomially in system size as $\mathcal{O}(\text{poly}(N))$.

C. Time-Evolving Block Decimation (TEBD)

When using the TEBD algorithm, we are interested in evaluating the time evolution of a quantum state

$$|\psi(t)\rangle = \hat{U}(t)\,|\psi(0)\rangle\,.$$

We consider a Hamiltonian with nearest-neighbor interactions and split it into even and odd indices

$$\hat{H} = \sum_{n\text{ odd}} \hat{h}^{[n,n+1]} + \sum_{n\text{ even}} \hat{h}^{[n,n+1]},$$

divide the time evolution operator into small time slices δt, and decompose the time-evolution operator by making use of the Suzuki-Trotter decomposition in linear order

$$\hat{U}(\delta t) = \prod_{n \text{ odd}} \hat{U}^{[n,n+1]}(\delta t) \prod_{n \text{ even}} \hat{U}^{[n,n+1]}(\delta t) + \mathcal{O}(\delta t^2). \tag{5}$$

This approximation makes it apparent that, due to the first-order nature, the Trotter error accumulates over long simulation times. Roughly speaking, the heart of the TEBD algorithm consists of the successive application of these two-site unitary operators, which locally update our MPS, i.e. quantum state, with every time step.[3]

D. Density Matrix Renormalization Group (DMRG)

DMRG is a variational algorithm used to find the ground state of a Hamiltonian $\hat{H}$ by optimizing a trial state $|\Theta\rangle$ in the form of a MPS. The MPS formalism can easily be expanded to matrix product operators (MPOs), allowing an efficient representation of the Hamiltonian. In each step, two neighboring sites n and $n+1$ are locally optimized to minimize the energy expectation value of the effective Hamiltonian $\langle\Theta| \hat{H}_{\text{eff}} |\Theta\rangle$. The key advantage over exact diagonalization lies in restricting the optimization to a much smaller effective Hamiltonian $\hat{H}_{\text{eff}}$, which can be diagonalized efficiently. One complete cycle of the algorithm is denoted as a "sweep". In one sweep, all neighboring sites of the 1D chain are being traversed and optimized from left to right - and back. An illustrative explanation of the DMRG algorithm can be seen in figure 1.[3]

II. RESULTS

A. Convergence of DMRG

We start our discussion by taking a closer look at the convergence of the DMRG algorithm for system size $L = 10$, $J = 1$ and multiple transverse-field strengths $g \in \{0.5, 1.0, 1.5\}$ by comparing the obtained ground state energy to the one obtained using exact diagonalization. More precisely, we would like to see how quick DMRG converges to the ground state energy by plotting the relative error $|(\hat{E}_0 - E_0)/\hat{E}_0|$ against the number of sweeps performed. Here, $\hat{E}_0$ denotes the energy obtained via exact diagonalization while E_0 is our DMRG estimate. The results are visualized in figure 2.

As we can see, DMRG achieves an accuracy of about 10^{-15} after just 3 sweeps for all different g's, showing its potential. In the following, we will choose the max number of sweeps to be 10 and allow early termination

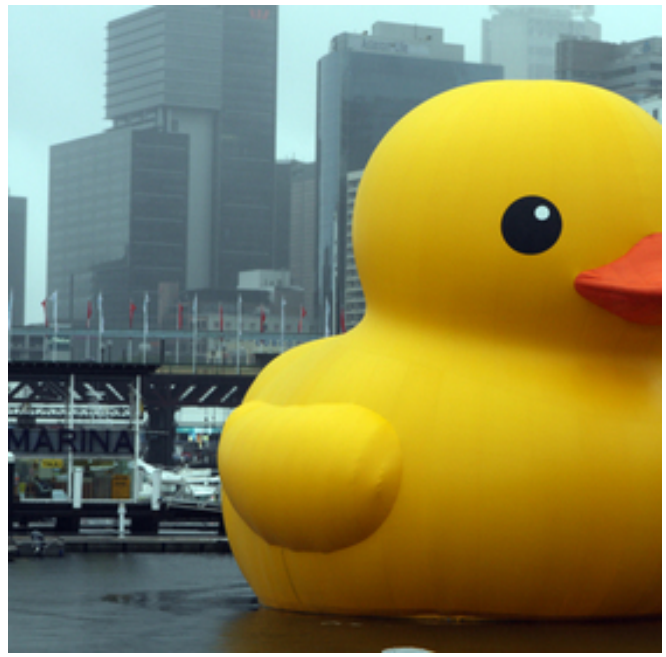

FIG. 1: (a) Tensor network diagram of the expectation value $E = \langle\psi| \hat{H} |\psi\rangle$. Parts to the left of site n (right of site $n+1$) are being contracted into an environment $L^{[n]}(R^{[n+1]})$. (b) Effective Hamiltonian $\hat{H}_{\text{eff}}$. (c) Update procedure of sites n and $n+1$. (d) Update rules for the environments according to (a). (**Figure and caption adapted from J. Hauschild and F. Pollmann [3]**)

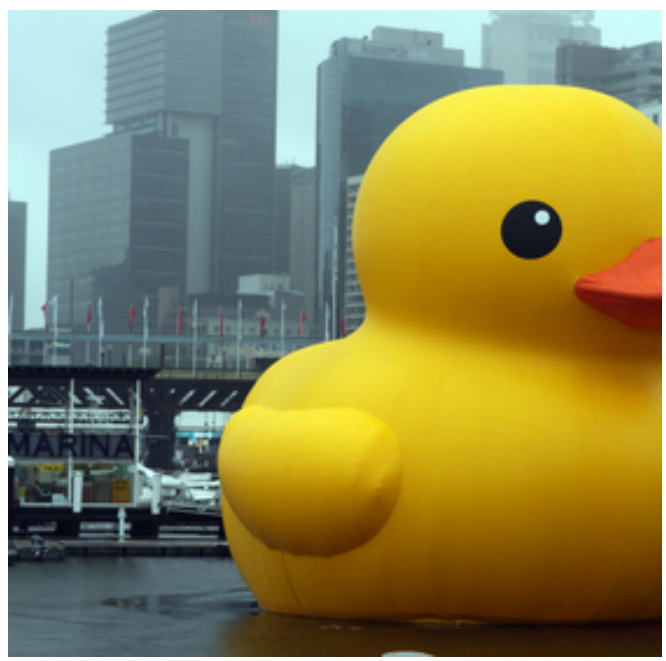

FIG. 2: Relative error of DMRG ground state energy E_0 with respect to energy obtained via exact diagonlization $\hat{E}_0$

.

of the algorithm, if the energy difference between the last and current sweep is less than 10^{-13}.

B. Convergence of TEBD

Knowing that DMRG provides an accurate ground state of the 1D TFI model, we can continue our discussion by analyzing the convergence of TEBD during the time evolution of a correlation function. TEBD convergence depends on the maximal bond dimension χ, as well as on the size of the time slices δt, and the total number of simulation steps N_{steps}.

We focus on the correlation function $\langle\psi|\,\hat{S}^{+}_{L/2}(t)\hat{S}^{-}_{L-1}(0)\,|\psi\rangle$, as it is among the most demanding in terms of the bond dimension required. We therefore assume it to set an upper bond. To determine suitable simulation parameters, we focus on the case $g = 0.5$. Even though we find the highest entanglement at the critical point $g = 1.0$, we find that the parameters obtained from $g = 0.5$ also yield reliable results.

From figure 3, we conclude that both sets of test parameters yield comparable results regarding TEBD convergence. So we stick with the computationally lighter choice: $\chi = 5, dt = 0.1$, and $N_{\text{steps}} = 100$.

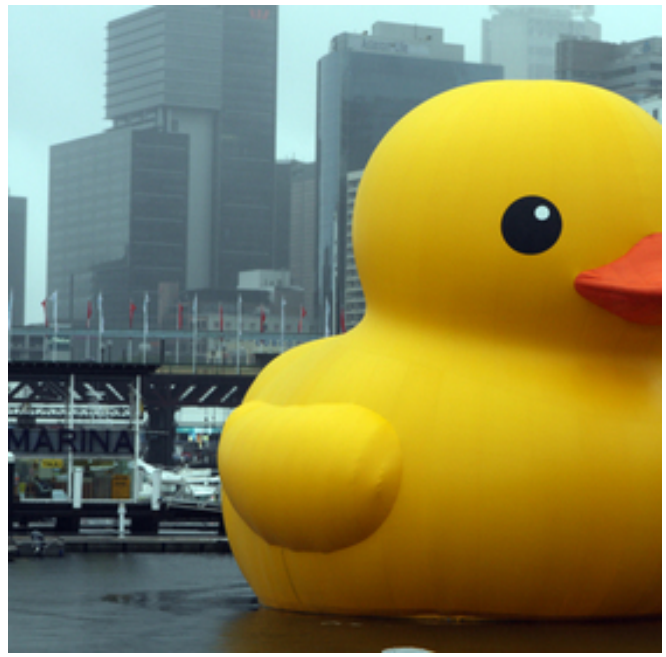

FIG. 3: TEBD convergence for $g = 0.5$ with parameter choices $(\chi_1 = 5, dt_1 = 0.1, N_{\text{steps, 1}} = 100)$ and $(\chi_2 = 10, dt_2 = 0.01, N_{\text{steps, 2}} = 1,000)$. The result from exact diagonalization is plotted as the dotted line.

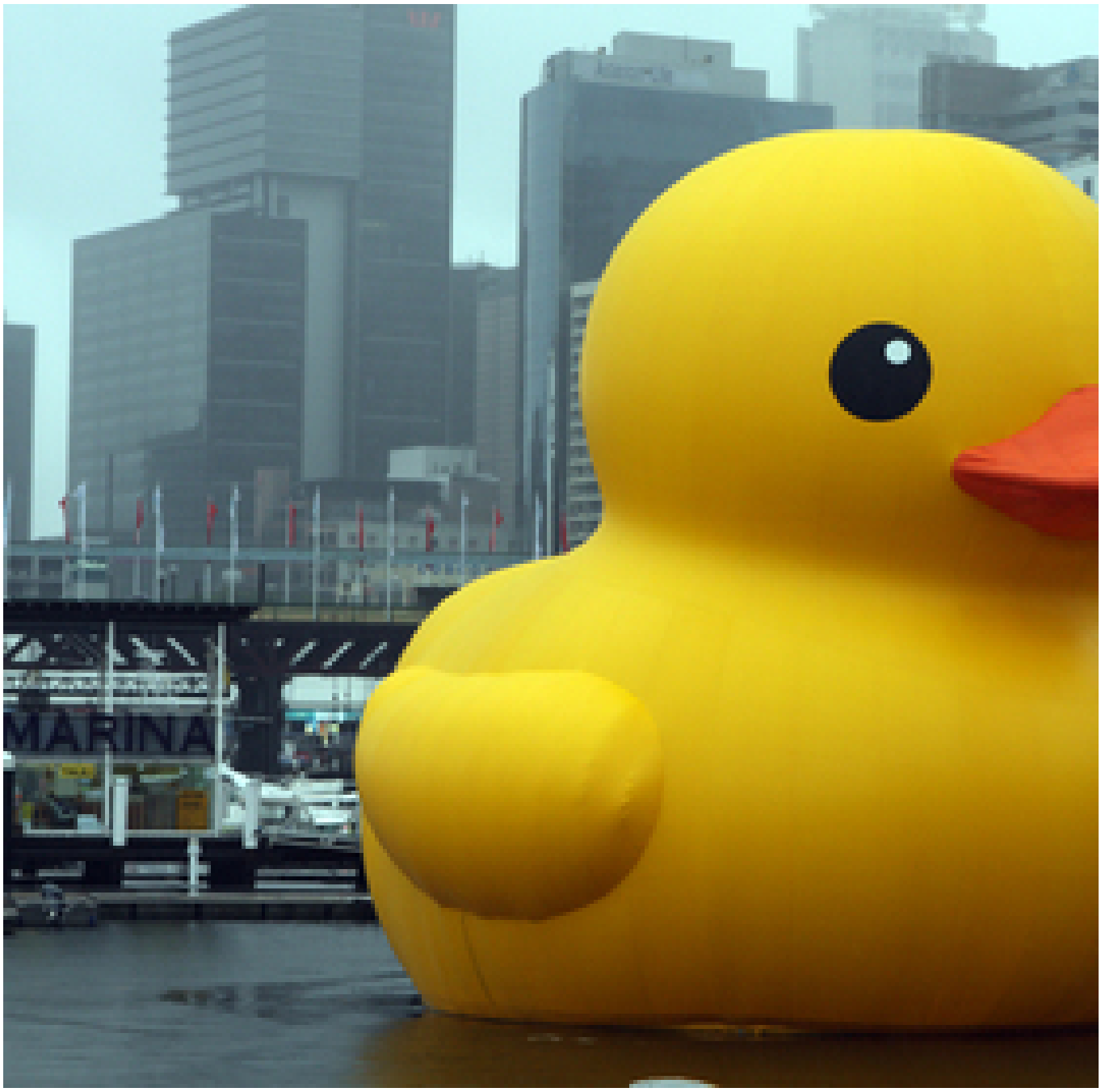

FIG. 4: TEBD vs ED for $g \in \{0.5, 1.0, 1.5\}$ with parameters $\chi = 5$, $dt = 0.1$, and $N_{\text{steps}} = 100$.

It is important to emphasize that the raising and lowering operators $\hat{S}_j^{\pm}$ are non-unitary. As a result, special attention must be given to maintain proper normalization of the time-evolved MPS. Additionally, the MPS looses its canonical form under generic non-unitary transformations - fortunately, the latter is not the case for the raising and lowering operators.

We conclude this subsection by showing how well TEBD approximates the result obtained via exact diagonalization, for all values of g. Figure 4 shows that the chosen parameters work well for all three cases and for times up to $N_{\text{setps}} \cdot dt = 10$.

C. Obtaining the spectral function

Now that we have sucessfully investigated both TEBD and DMRG and compared them to the corresponding ED results, we are finally ready to compute the associated spectral function $A(\omega, k)$. This is done by performing a

discrete Fourier transformation in both space an time:

$$A(\omega, k) = \sum_{\Delta L=-L+1}^{L-1} \sum_{t=0}^{T} \langle \hat{S}^{+}_{L/2}(t) \hat{S}^{-}_{L/2+\Delta L}(0) \rangle e^{-i(\omega t + k\Delta L)}$$

where we cover all spatial correlations and all simulation times up to $T = 10$. To enhance resolution in the k domain, we increase the system size to $L = 30$, which would be infeasible using ED. A bond dimension of $\chi = 75$ turned out to be sufficient to yield a comprehensive visual result, as shown in figure 5.

FIG. 5: Obtained absolute values of the spectral function $A(\omega, k)$ for $g \in \{0.5, 1.0, 1.5\}$.

Unfortunately, the ω-axis shows an incorrect scaling and offset. Despite best efforts - including the use of filters and subtraction of disconnected components from the correlation functions - the issue has not been resolved.

As a result, the expected closing of the energy gap at $g = 1.0$ is not visible in the final result.

But we can still gain valuable insights from the spectral weight: At $g = 0$ the system is fully ordered and we observe a single, sharp excitation at $k = 0$ without any further dispersion. At the critical point $g = 1.0$, the spectral weight spreads across the energy band, as a consequence of quantum fluctuations, enabling more transitions. Finally, at $g = 1.5$ the system is in a disordered phase that shows an intensified and well-defined band structure, mirroring the shape of the quasiparticle dispersion from eq. 2.

III. CONCLUSION

Within this work, it was demonstrated that for a system size for $L = 10$ and various values of g, DMRG provides accurate ground states in agreement with ED. Additionally, for the same system parameters, TEBD reproduced an accurate time-evolution for a bond dimension of $\chi = 5$, time slices $dt = 0.1$, and $N_{\text{steps}} = 100$, again in comparison to ED. The system size was then increased to $L = 30$, in order to enhance the resolution in the k domain. The spectral weights match our expectations for all values of g. The scaling and offset of the ω-axis remains an unresolved issue that should be investigated on further to recover the exact quasiparticle dispersion.

[1] A. Dutta, G. Aeppli, B. K. Chakrabarti, U. Divakaran, T. F. Rosenbaum, and D. Sen, Quantum phase transitions in transverse field spin models: from statistical physics to quantum information (2015), arXiv:1012.0653 [cond-mat.stat-mech].

[2] P. Coleman, *Introduction to Many-Body Physics* (Cambridge University Press, 2015).

[3] J. Hauschild and F. Pollmann, Efficient numerical simulations with tensor networks: Tensor network python (tenpy), SciPost Physics Lecture Notes 10.21468/scipost-physlectnotes.5 (2018).

Appendix A: Code listing

```python
# Dependencies
import numpy as np
import matplotlib.pyplot as plt
import pickle
from scipy.sparse import csr_matrix, kron
from scipy.sparse.linalg import expm
from copy import deepcopy

import a_mps
import b_model_mod # only slight modifications were made
import c_tebd
import d_dmrg
import tfi_exact_mod # only slight modifications were made

L = 10
J = 1
g = 0.5
g_vals = [0.5, 1.0, 1.5]
models_DMRG = {}
psis_DMRG = {}
max_N_sweeps = 10
eps = 1.e-13 # Use this as precision


class Sigma():
    x = np.array([[0., 1.], [1., 0.]])
    y = np.array([[0., -1j], [1j, 0.]])
    z = np.array([[1., 0.], [0., -1.]])
    plus = np.array([[0., 1.], [0., 0.]])
    minus = np.array([[0., 0.], [1., 0.]])

# Function to run DMRG to determined accuracy
def get_groundstate_DMRG(L, J, g, max_N_sweeps = 10, chi_max = 30, eps = 1e-10, terminate_early=True):
    psi = a_mps.init_spinup_MPS(L)
    model = b_model_mod.TFIModel(L, J, g)
    engine = d_dmrg.DMRGEngine(psi=psi, model=model, chi_max=chi_max, eps=eps)
    energy = model.energy(psi)
    energy_old = energy + 2*eps
    print(f"Running DMRG simulation for L = {L}, g = {g}, J = {J}, eps = {eps}, max_N_sweeps = {max_N_sweeps}")
    i = 1
    while (np.abs((energy_old - energy)/energy) >= eps and terminate_early) or max_N_sweeps >= i:
        energy_old = energy
        engine.sweep()
        energy = model.energy(psi)
        i += 1
        #print("   Rel. energy difference to last iteration:", np.abs((energy-energy_old)/energy))
    print(">> Done.")
    return model, psi

# Normalizes the MPS
def normalize_mps(mps: a_mps.MPS) -> a_mps.MPS:
```

```python
    mps_new = mps.copy()
    norm = np.sqrt(mps_overlap(mps_new, mps_new))
    for i in range(mps_new.L):
        mps_new.Bs[i] = mps_new.Bs[i] / norm
    return mps_new

# Multiplies operator op to mps psi on site j
def _matvec(op, psi: a_mps.MPS, j: int) -> a_mps.MPS:
    new_psi = deepcopy(psi)
    B = new_psi.Bs[j]
    B_new = np.tensordot(op, B, axes=(1, 1))
    B_new = np.transpose(B_new, (1, 0, 2))
    new_psi.Bs[j] = B_new
    return new_psi

# Function to calculate the overlap of two MPS
def mps_overlap(bra: a_mps.MPS, ket: a_mps.MPS):
    assert bra.L == ket.L
    L = bra.L
    env= np.array([[1.0]]) # vR vR*
    for n in range(L):
        B_bra = bra.Bs[n].conj() # vL* i* vR*
        B_ket = ket.Bs[n] # vL i vR
        env = np.tensordot(env, B_ket, axes=(0, 0)) # vR* i vR
        env = np.tensordot(env, B_bra, axes=[(0, 1), (0, 1)]) # vR vR*
    return env.item()

# Calculates correlation functions of the form <psi| Op1(t) Op2(0) |psi>
# This method also time evolves the state
def calc_correlations(model, psi, op1, op2, i, j, dt=0.1, N_steps=100, chi_max=30, eps=1e-10):
    # Raising and lowering operator do not preserve canonical form, nor norm!
    # After _matvec multiplication, the reconstruction of the canonical form and normalization is generally required!
    print("Calculating Dynamical correlation functions...")
    print(f"System Size: {model.L}, Total time: {dt * N_steps}")
    correlations = {}
    U_bonds = c_tebd.calc_U_bonds(model, 1.j * dt)
    ket = deepcopy(psi)
    ket = _matvec(op2, ket, j)
    norm_correction = np.sqrt(mps_overlap(ket, ket))  # ||S+|psi>||
    for step in range(N_steps + 1):
        t = step * dt
        print(f"  t = {t}")
        ket_tmp = deepcopy(ket)
        ket_tmp = _matvec(op1, ket_tmp, i)
        val = mps_overlap(psi, ket_tmp)
        if step > 0:
            val *= norm_correction
        correlations[t] = val
        # Time evolve for next step
        if step < N_steps:
            c_tebd.run_TEBD(ket, U_bonds, 1, chi_max, eps)
    return correlations

# Time Evolution for Exact Diagonlization
def time_evlove_ED(L=L, J=J, g=g, op1=csr_matrix(Sigma.plus), op2=csr_matrix(Sigma.minus), i=L//2, j=L-1, dt=0.5, T_max=5):
    print(f"Starting ED with L={L}, J={J}, g={g}")
    id = csr_matrix(np.eye(2))
    def get_ops(op: csr_matrix, n, L): # site index n: 0, ..., L-1
        full = op if 0 == n else id
        for l in range(1, L):
            full = kron(full, op if l == n else id)
        return full
    energy, psi0, H = tfi_exact_mod.calc_ED(L, J, g)
    op1s = get_ops(op1, i, L)
    op2s = get_ops(op2, j, L)
    print(">> Done")
    # Time evolve
    correlations = {}
```

```python
    for t in np.arange(0, T_max+dt, dt):
        print(f"..Computing time step {t}")
        psi_evolved = expm(1.j*H*t)@op1s@expm(-1.j*H*t)@op2s@psi0
        correlations[t] = psi0.conj().T @ psi_evolved

    print(">> Done")
    return correlations

# Obtain the spectral function given a dictionary correlations[DeltaLs][ts]
# This method still shows an unresolved scaling issue / offset for the omega-axis.
def get_spectral(correlations):
    DeltaLs = sorted(correlations.keys())
    L = len(DeltaLs)
    times = sorted(correlations[DeltaLs[0]].keys())
    N = len(times)
    dt = times[1] - times[0]
    corr_array = np.zeros((L, N), dtype=complex)
    for i, dL in enumerate(DeltaLs):
        for j, t in enumerate(times):
            corr_array[i, j] = correlations[dL][t]
    # The commented lines don't provide the correct scaling either
    #A_omega = np.fft.fftshift(np.fft.fft(corr_array, axis=1), axes=1)
    #omega_vals = np.fft.fftshift(np.fft.fftfreq(N, dt)) * 2 * np.pi
    A_omega = np.fft.fft(corr_array, axis=1)
    omega_vals = np.fft.fftfreq(N, dt) * 2 * np.pi
    A_k_omega = np.fft.fftshift(np.fft.fft(A_omega, axis=0), axes=0)
    k_vals = np.fft.fftshift(np.fft.fftfreq(L, d=1)) * 2 * np.pi

    return A_k_omega, k_vals, omega_vals

#Take absolute value of correlation for better comparison
def get_absolute_correlations(correlations_XY):
    return {
        g: {t: np.abs(val) for t, val in inner_dict.items()}
        for g, inner_dict in correlations_XY.items()
    }

#%%
L = 10
J = 1
g = 0.5
g_vals = [0.5, 1.0, 1.5]
models_DMRG = {}
psis_DMRG = {}
max_N_sweeps = 10
eps = 1.e-13 # Use this as precision
# Determine Accuracy of DMRG
E0_ED = {}
H_ED = {}
for g in g_vals:
    print(f"Computing for g = {g}")
    models_DMRG[g] = {}
    psis_DMRG[g] = {}
    for N in range(max_N_sweeps):
        print(f"..DMRG sweep {N} out of {max_N_sweeps}")
        models_DMRG[g][N], psis_DMRG[g][N] = get_groundstate_DMRG(L, 1, g, chi_max=50, max_N_sweeps=N, terminate_early=False)
        print("..>> Done")
    E0_ED[g], _, H_ED[g]  = tfi_exact_mod.calc_ED(L, 1, g)
    print(">> Done")
# Plot the results
fig, ax = plt.subplots(figsize=(3.4, 3.4))
ax.set_xlabel("# sweeps")
ax.set_ylabel(r"Rel. Error")
ax.grid(True)
ax.set_yscale("log")
for g in g_vals:
    energies = [models_DMRG[g][key].energy(psis_DMRG[g][key]) for key in models_DMRG[g].keys()]
    rel_error = np.abs(np.array(energies) - E0_ED[g]) / np.abs(E0_ED[g])
    ax.plot(models_DMRG[g].keys(), rel_error, '--o', label=f"$L = {L}$, $J = 1$, $g ={g}$")
```

```python
ax.legend(loc = 'best')
plt.tight_layout()
#plt.savefig("ProjectMaterials/RelErrorSweeps.pdf")
plt.show()
# %%
# Determine parameters for time TEBD, first: g=0.5 alone

# Instead of running the simulation again, we can simply open the pickle file
#correlations_ED = {}
#for g in g_vals:
#    correlations_ED[g] = time_evlove_ED(L=L, J=1, g=g, dt = 0.2, T_max= 10)
#    with open(f'ProjectMaterials/corr_ED_g{g}.pkl', 'wb') as f:
#        pickle.dump(correlations_ED[g], f)
correlations_ED = {}
for g in g_vals:
    with open(f'ProjectMaterials/corr_ED_g{g}.pkl', 'rb') as f:
        correlations_ED[g] = pickle.load(f)
correlations_ED_abs = get_absolute_correlations(correlations_ED)
colors = {0.5: 'tab:blue', 1.0: 'tab:orange', 1.5: 'tab:green'}

chis = [5, 10]
dts = [0.1, 0.01]
N_steps_vals = [100, 1000]

fig, ax = plt.subplots(figsize=(3.4, 3.4))
ax.set_xlabel("time t")
ax.set_ylabel(r"$| \langle S_{L/2}^+ (t)  S_{L-1}^- (0) \rangle|$")
ax.grid(True)
ax.plot(correlations_ED_abs[0.5].keys(), correlations_ED_abs[0.5].values(), '--+', c=colors[0.5], label=f'ED ($g=0.5$)')
#ax.plot([0], [0.24], c='white', label=r"($\chi, dt, \text{# steps}$)")
for chi in chis:
    for dt, N_steps in zip(dts, N_steps_vals):
        model_FM, psi_FM = get_groundstate_DMRG(L=L, J=1, g=0.5, chi_max=chi)
        correlations_FM = calc_correlations(model_FM, psi_FM, Sigma.plus, Sigma.minus, i = L//2, j=L-1, dt=dt, N_steps=N_steps, chi_max=chi)
        correlations_FM_abs = {
           k: np.abs(val) for k, val in correlations_FM.items()
        }
        #ax.plot(correlations_FM_abs.keys(), correlations_FM_abs.values(), label=f"({chi}, {dt}, {N_steps})")
        ax.plot(correlations_FM_abs.keys(), correlations_FM_abs.values())
ax.legend()
plt.tight_layout()
#plt.savefig("ProjectMaterials/FMTEBD.pdf")
plt.show()
# %%
# Lets include all values of g
models_DMRG = {}
for g in g_vals:
    models_DMRG[g], psis_DMRG[g] = get_groundstate_DMRG(L, 1, g, chi_max=5)
correlations_DMRG ={}
for g in g_vals:
    correlations_DMRG[g] = calc_correlations(models_DMRG[g], psis_DMRG[g], Sigma.plus, Sigma.minus, L//2, L-1, dt=.1, N_steps=100, chi_max=5, eps=eps)

correlations_DMRG_abs = get_absolute_correlations(correlations_DMRG)

relative_errors = {
    g: {
        t: abs(correlations_DMRG_abs[g][t] - correlations_ED_abs[g][t]) / abs(correlations_ED_abs[g][t])
        for t in correlations_ED_abs[g]
    }
    for g in g_vals
}



fig, ax = plt.subplots(nrows=2, sharex=True, figsize=(6.8, 6.8))
```

```
for g in g_vals:
    ax[0].plot(correlations_DMRG_abs[g].keys(), correlations_DMRG_abs[g].values(), c=colors[g], label=f'TEBD ($g={g}$)')
    ax[0].plot(correlations_ED_abs[g].keys(), correlations_ED_abs[g].values(), '--+', c=colors[g], label=f'ED ($g={g}$)')
    ax[1].plot(relative_errors[g].keys(), relative_errors[g].values(), '--o', c=colors[g], label=f"g={g}")
ax[1].set_xlabel("time t")
ax[0].set_ylabel(r"$| \langle S_{L/2}^+ (t)  S_{L-1}^- (0) \rangle|$")
ax[0].grid()
ax[0].set_title("TEBD vs. ED time evolution")
ax[0].legend()
ax[1].grid()
ax[1].set_ylabel("Rel. Error")
ax[1].legend()
plt.tight_layout()
#plt.savefig("ProjectMaterials/TEBDvsED.pdf")
plt.show()
# %%
# Plot spectral functions

#L = 30
#J = 1
#chi_max = 75 # raise chi significantly
#correlations = {}
#for g in g_vals:
#    model_DMRG, psi_DMRG = get_groundstate_DMRG(L, J, g, chi_max=chi_max, max_N_sweeps=max_N_sweeps)
#    correlations[g] = {}
#    for j in range(0, L):
#        DeltaL = j - L//2
#        if DeltaL not in correlations[g]:
#            correlations[g][DeltaL] = calc_correlations(model_DMRG, psi_DMRG, Sigma.plus, Sigma.minus, L//2, j, dt=0.1, N_steps=100, chi_max=chi_max, eps=eps)
#
#with open(f'ProjectMaterials/corr_Spectral.pkl', 'wb') as f:
#    pickle.dump(correlations, f)

with open(f'ProjectMaterials/corr_Spectral.pkl', 'rb') as f:
    correlations = pickle.load(f)

def get_spectral(correlations):
    DeltaLs = sorted(correlations.keys())
    L = len(DeltaLs)
    times = sorted(correlations[DeltaLs[0]].keys())
    N = len(times)
    dt = times[1] - times[0]

    corr_array = np.zeros((L, N), dtype=complex)
    for i, dL in enumerate(DeltaLs):
        for j, t in enumerate(times):
            corr_array[i, j] = correlations[dL][t]

    #A_omega = np.fft.fftshift(np.fft.fft(corr_array, axis=1), axes=1)
    #omega_vals = np.fft.fftshift(np.fft.fftfreq(N, dt)) * 2 * np.pi
    A_omega = np.fft.fft(corr_array, axis=1)
    omega_vals = np.fft.fftfreq(N, dt) * 2 * np.pi
    A_k_omega = np.fft.fftshift(np.fft.fft(A_omega, axis=0), axes=0)
    k_vals = np.fft.fftshift(np.fft.fftfreq(L, d=1)) * 2 * np.pi

    return A_k_omega, k_vals, omega_vals

A = {}
k_vals = {}
omega_vals = {}
for g in g_vals:
    A[g], k_vals[g], omega_vals[g] = get_spectral(correlations[g])
fig, ax = plt.subplots(ncols= 3, figsize=(8, 3.4), sharey=True, sharex=True)
ims = []
for i, g in enumerate(g_vals):
```

```
    im = ax[i].imshow(np.abs(A[g].T), extent=[k_vals[g][0], k_vals[g][-1], omega_vals[g][0], omega_vals[g][-1]],
           origin='upper', aspect='auto')
    ax[i].set_title(f"$g={g}$")
    ax[i].set_xlabel(r'$k$' if i == 1 else '')
    ax[i].set_ylabel(r'$\omega$' if i == 0 else '')
    ims.append(im)

plt.tight_layout()

cbar = fig.colorbar(ims[0], ax=ax, orientation='vertical', fraction=0.3, pad=0.04)
cbar.set_label(r"$|A(\omega, k)|$")
#fig.savefig("ProjectMaterials/Spectral.pdf")
plt.show()
```